ADrive 리니어 액추에이터 퀵스타트 (SDK 0.1)

1. 준비물

항목	내용
PC	Windows 10/11 64-bit, Python 3.10 이상 (3.10–3.13 에서 같은 wheel 하나를 쓴다)
액추에이터	ADrive-LH42 + 48 V 직류 전원
연결	USB 케이블(기본) 또는 CAN 어댑터 — 4 장 참고
안전	**전원 차단형 비상정지 스위치**(필수). SDK 의 `estop()` 은 소프트웨어 정지이고 안전 등급이 아니다
고정	액추에이터 본체를 바이스·클램프·볼트로 고정한다. 양면테이프 고정은 금지(가속 시 이탈)

2. 설치

```
pip install adrive-0.1.0-cp310-abi3-win_amd64.whl
python -c "import adrive; print(adrive.build_info())"
```

- **드라이버 설치가 필요 없다.** 액추에이터는 MS OS 디스크립터(호환 ID `WINUSB`)를 제공해서 Windows 8.1 이후에서 WinUSB 로 자동 인식된다(Zadig 불필요).
- 장치 관리자에서 COM 포트가 하나 같이 보일 수 있다. SDK 는 그 포트를 쓰지 않는다.
- `build_info()` 의 `build_sha` 는 지원 문의 때 알려 주면 된다.

3. 처음 연결 — 호밍은 반드시 먼저

전원을 켜기 전에 캐리지를 스트로크 중앙 부근, 양 끝에서 떨어진 위치에 둔다. 전원을 켜면 액추에이터가 스스로 짧게 움직이며 엔코더 인덱스를 찾고 엔코더를 맞춘다(정상. 8 장 7 번 참조). 그 뒤 **호밍(원점 찾기)을 한 번 해야 이동 명령이 허용된다.** 호밍은 수축 방향 끝의 엔드스톱 스위치까지 2 mm/s 로 천천히 움직이고, 그 점을 0 mm 로 잡는다.

```
import adrive

print(adrive.scan())                # ['usb:<serial>']  — 연결된 액추에이터 목록
with adrive.connect() as act:       # 1 대면 자동 선택. 여러 대면 connect("usb:<시리얼>")
    print(act.info())               # 모델, 시리얼, 펌웨어
    act.home()                      # 전원 투입 후 1 회 필수 (최대 60 s)
    act.enable()                    # 현재 위치를 유지하며 제어 시작
    act.move_to(10.0)               # 10 mm 로 이동 (기본 20 mm/s, 100 mm/s²)
    print(act.position())           # mm
    act.move_to(3.0, speed=50, accel=500)
# with 블록을 나가면 모터는 자동으로 꺼진다(disable)
```

- 단위는 항상 **mm, mm/s, mm/s², N, s** 다.
- 이동 범위(소프트 리밋)는 기본 0.5–25 mm 다. `act.set_soft_limits(2, 20)` 처럼 좁힐 수만 있고 넓힐 수는 없다.
- 속도·가속 상한은 200 mm/s, 2000 mm/s² 다. 생략하면 그 10 % 를 쓴다.
- `move_to(..., wait=True)`(기본)은 목표 ±10 µm 에 들어가거나, 궤적이 끝나고 0.5 s 동안 멈춰 있으면 반환한다. 마찰 때문에 수 µm–수십 µm 가 남을 수 있으니 `position()` 으로 확인한다.

4. USB / CAN 자동 인식

> **CAN 으로는 아직 움직일 수 없다 (현 펌웨어).** 현 펌웨어는 CAN 호밍을 지원하지 않아 SDK 가 거부한다. SDK 는 호밍하지 않은 축의 이동도 거부한다. 따라서 CAN 축은 찾기·상태 보기·정지만 된다. 이후 펌웨어에서 CAN 이동이 지원될 때까지 **이동은 USB 로 한다.** 실제 CAN 어댑터는 이 배포에서 하드웨어로 시험하지 않았다(미검증).

연결	방법	상태
USB	`adrive.connect()` / `connect("usb:<시리얼>")` — 자동 인식	지원
CAN	자동 인식(하트비트) 또는 `connect("can:<백엔드>:<채널>:<노드 ID>")`	찾기·상태·정지만. 이동 불가(위 참고). 실 어댑터 미검증

지원 CAN 어댑터

어댑터	백엔드	타깃 예	주의
PEAK PCAN-USB	`pcan`	`can:pcan:usb1:16`	PEAK 의 PCAN-Basic 드라이버(DLL)를 먼저 설치한다
candleLight / gs_usb (예: candleLight 펌웨어의 CANable)	`gsusb`	`can:gsusb:0:16`	WinUSB 드라이버가 필요하다
slcan (CANable 기본 펌웨어)	`slcan`	`can:slcan:COM5:16`	COM 포트로 보인다
Linux SocketCAN	`socketcan`	`can:socketcan:can0:16`	인터페이스는 미리 1 Mbit/s 로 올려 둔다

- 비트레이트는 **1 Mbit/s** 다.
- 자동 인식은 드라이브 하트비트를 약 1.6 s 동안 듣고 축을 찾는다. 축은 CAN 노드 ID 로 구분한다(기본 16 = 0x10).
- CAN 으로는 시리얼 번호와 펌웨어 버전을 알 수 없다. 노드 ID 만 보인다.
- 같은 버스에 노드 ID 가 겹치면 그 ID 는 경고를 내고 자동 인식에서 뺀다(`can:...` 로 직접 지정하면 연결은 된다). 여러 대를 한 버스에 둘 때는 노드 ID 를 모두 다르게 한다.
- CAN 탐색을 건너뛰려면 환경 변수 `ADRIVE_CAN_SCAN=0` 을 설정한다.
- 여러 대: `adrive.connect_all()` 은 USB 로 연결된 모든 액추에이터와 자동 인식으로 찾은 CAN 축을 연다(`ADRIVE_CAN_SCAN=0` 이면 CAN 제외). 찾지 못한 CAN 노드는 `connect_all(["can:pcan:usb1:16", "can:pcan:usb1:17"])` 처럼 직접 추가한다.

5. 힘 제어

```
act.force_control(10.0, max_speed=2.0)      # +10 N 으로 민다, 속도는 2 mm/s 이하
```

- **명령 유효기간(lease)**: 힘 명령은 기본 1 s 동안만 유효하다. 계속 밀려면 그 안에 `force_control` 을 다시 불러야 한다. 스크립트가 멈추거나 오류로 빠지면 액추에이터는 그 자리에서 위치를 유지한다.

```
end = time.time() + 5
while time.time() < end:
    act.force_control(10.0, max_speed=2.0, lease=0.5)
    print(act.force())                      # N (추정치)
    time.sleep(0.1)
act.stop()                                  # 현재 위치 유지
```

- `force()` 는 모터 전류로 계산한 **추정치**다. 스크루 마찰이 보정되지 않아 수십 N 오차가 날 수 있다. 정밀한 힘(±1 N)은 로드셀이 필요하다. 힘 정밀도는 아직 로드셀로 검증하지 않았다.
- 힘 모드에서 소프트 리밋을 넘으면 그 위치에서 멈추고 위치 유지로 바뀐다. 아무것도 밀지 않는 상태(무부하)에서 힘을 주면 속도 상한까지 움직인 뒤 리밋에서 멈춘다.

6. 4 축 동기 제어 예제

```
python examples/sync_4axis.py                   # USB 로 연결된 모든 축
```

```
python examples/sync_4axis.py --extra can:pcan:usb1:16   # CAN 노드 추가 (현재는 이동 불가 — 4 장)
python examples/sync_4axis.py --sim 3              # 하드웨어 없이 시뮬레이터 축 추가
```

모든 축을 동시에 호밍한 뒤, 축마다 다른 거리를 같은 시간에 이동한다. 이동마다 시작 동기 오차(명령·실제 출발 시각의 차), 축 간 위상 오차(중간점 통과 시각의 차), 이상 프로파일 대비 최대 편차를 출력한다. 한 축이라도 에러가 나면 모든 축을 즉시 정지한다.

현 펌웨어에서는 CAN 축이 하나라도 섞이면 그 축의 호밍이 거부되고, 예제는 모든 축을 정지한 뒤 끝난다(4 장). 지금은 USB 축으로만 이동한다. CAN 배선·어댑터별 주의는 `examples/sync_4axis.py --help` 에 있다.

7. GUI

Windows 용 단일 실행 파일 `ADrive-GUI.exe` 를 제공한다. 설치 없이 실행한다. Tk 로 만들었고 공개 SDK 만 쓴다.

화면 / 기능	내용
장치 목록	USB·CAN 자동 인식. 시리얼, 모델, 펌웨어, 링크를 보여 준다
연결 / 해제	목록에서 골라 연결한다
호밍	Home 버튼과 호밍 상태 배지. 호밍 전에는 이동 버튼이 꺼져 있다
Enable / Disable	모터 제어 켜기·끄기
이동	목표 mm 입력, 프리셋(소프트 리밋 범위의 0–100 %), 조그 ±0.01 / 0.1 / 1 / 10 mm
속도·가속	슬라이더. SDK 상한을 넘지 않는다
힘 탭	**EXPERIMENTAL**(로드셀 시험 전). 명령 유효기간(lease)을 쓴다
그래프	위치, 목표, 전류, 힘 추정치 실시간 표시
에러 표시	드라이브 에러를 보여 준다
EMERGENCY STOP	큰 버튼, 단축키 Space / Esc. 모든 축의 모터를 끈다(idle)
CSV 기록	측정값을 CSV 로 저장한다
다축	최대 4 축. 동기 이동을 지원한다

- GUI 의 EMERGENCY STOP 은 **소프트웨어 정지**다. 안전 등급 비상정지가 아니다. 전원 차단 비상정지 스위치는 따로 둔다.
- 4 장의 CAN 제한은 GUI 에도 똑같이 적용된다.
- 한 번에 한 프로그램만 장치를 열 수 있다. GUI 를 쓰는 동안 다른 파이썬 스크립트로 같은 장치를 열지 않는다.
- 소스에서 하드웨어 없이 시뮬레이터 4 축으로 실행: `python gui/adrive_gui.py --sim 4`

8. 안전 수칙

1. 처음 움직일 때는 낮은 속도(≤ 10 mm/s)와 짧은 거리(1–5 mm)로 방향을 확인한다. 양(+) 방향 = 엔드스톱에서 멀어지는 쪽(신장).
2. 사람의 손·몸이 로드 끝 이동 범위 안에 들어가지 않게 한다. 전원 차단 비상정지를 손 닿는 곳에 둔다.
3. **열 보호 주의**: 이 버전은 드라이버 온도 감시가 동작하지 않는다(측정값 무효). 고속 왕복을 연속으로 오래 돌리지 말고, 짧게 돌린 뒤 쉬는 시간을 둔다.
4. 스트로크 끝(0 mm 쪽 엔드스톱, 25 mm 쪽 기구 끝)에 부딪히지 않게 소프트 리밋을 쓴다.
5. 프로그램이 죽으면 0.5 s 안에 모터가 꺼진다(통신 감시). 꺼진 모터는 힘을 내지 않으므로, 수직 설치나 외력이 있는 구조에서는 축이 밀릴 수 있다.
6. 펌웨어·설정 변경 도구는 제공하지 않는다. 분해·개조하지 않는다.
7. **전원 인가 직후 엔코더 탐색**: 전원을 인가하면 드라이브가 스스로 엔코더 인덱스를 찾고 캘리브레이션하며, 이때 캐리지가 짧게 움직인다. 전원을 켜기 전에 캐리지를 양 끝에서 15 mm 이상 떨어뜨리고, 엔드스톱·바이스·지그·부하에 걸리지 않게 한다. 걸리면 이동량이 모자라 드라이브가 `ENCODER_FAILED`(0x100), 엔코더 오류 `CPR_POLEPAIRS_MISMATCH`(0x2)를 보고하고 모터는 꺼진 채로 남는다(9 장 참조).

9. 문제 해결

증상	원인 / 조치
`NotFoundError`	USB 케이블·전원 확인. `adrive.scan()` 이 빈 목록이면 장치 관리자에서 인식 여부 확인
`LinkError: E102 device busy`	다른 프로그램(다른 파이썬, GUI)이 이미 장치를 열고 있다. 한 번에 한 프로그램만 연결할 수 있다
`SafetyError: not homed`	전원 투입 후 `home()` 을 먼저 실행
`SafetyError: ... outside soft limits` / speed	목표·속도가 허용 범위 밖. 메시지의 범위 안으로
`FaultError (code ...)`	액추에이터가 보호 정지했다. `disable()` → `clear_faults()` → 원인 확인 후 `enable()`. 코드는 지원 문의에 적는다
프로그램이 죽은 뒤 다음 연결에서 `FaultError`	통신 감시가 모터를 끈 기록이다. `clear_faults()` 로 지운다
`TimeoutError` (move_to)	목표 근처에서 마찰로 천천히 다가가는 중일 수 있다. `timeout=` 을 늘리거나 `wait=False` 후 `position()` 확인
`LinkError ... reconnect required`	통신이 끊겼다. 케이블 확인 후 `connect()` 를 다시 한다(자동 재연결은 하지 않는다 — 의도적)
전원 인가 직후 엔코더 오류 `CPR_POLEPAIRS_MISMATCH`(0x2) / `ENCODER_FAILED`(0x100), 모터가 꺼진 채	전원 인가 직후 엔코더 탐색 구간에서 캐리지가 걸려(엔드스톱에 붙어 있음, 바이스·지그 간섭, 부하 연결) 실제 이동량이 모자란 경우다. 전원을 끄고, 캐리지를 손으로 엔드스톱에서 15 mm 이상 떨어뜨리고 걸림을 제거한 뒤 전원을 다시 켠다. 재발하면 엔코더 커넥터를 확인한다. 저장된 설정값 문제가 아니다

10. 라이선스 고지

- SDK·GUI·문서·액추에이터 펌웨어는 AeiROBOT 독점 소프트웨어다. 사용 허락 범위는 별도 계약(EULA)을 따른다.
- SDK 가 쓰는 제 3 자 라이브러리(Rust 크레이트: pyo3, nusb, serde_json 등, MIT/Apache-2.0)의 고지는 `THIRD_PARTY_NOTICES.txt` 로 함께 배포한다.